

ADVANCED DATA ANALYSIS WITH PYTHON

Session 3: Advanced Functions and Functional Programming

Arbitrary arguments, lambda expressions, docstrings, annotations, and list tools

Silvia Tulli

Today's agenda

01

Recap & warm-up

Quick review of last session, and any questions

02

Arbitrary argument lists

**args and **kwargs, and how they combine with normal parameters*

03

Lambda expressions

Anonymous functions, and using them as key= arguments

04

Docstrings & function annotations

Documenting functions, and annotating parameter/return types

05

Tools for working with lists

map, filter, comprehensions, and sorted() with a key

06

Wrap-up & practice

What to try before session 4

Wooclap Recap



Recap: control flow & comprehensions

CONTROL FLOW

- `pass` does nothing:
it's a placeholder, not a `skip`
- `continue` jumps to the next iteration;
`break` exits the loop entirely
- Chained comparisons,
e.g. `10 <= x <= 20`

LOOPING & COMPREHENSIONS

- `in` / `not in` test membership in a list or string
- `zip(a, b)` pairs elements; `dict.items()` gives key, value
- List comprehension:
`[expr for item in iterable if cond]`

Before class: today's readings

01	Arbitrary Argument Lists	Collecting extra positional and keyword arguments controlflow.html#arbitrary-argument-lists
02	Lambda Expressions	Small anonymous functions written inline controlflow.html#lambda-expressions
03	Documentation Strings	Conventions for documenting what a function does controlflow.html#documentation-strings
04	Function Annotations	Optional metadata about parameter/return types controlflow.html#function-annotations
05	Tools for Working with Lists	map, filter, comprehensions, and other list helpers stdlib2.html#tools-for-working-with-lists

READINGS FOR TODAY

Notebook

[Students' Notebook \(no solutions\)](#)

READINGS FOR TODAY

AI tools deep dive

github.com/cg67qowo/Advanced-Python/tree/main/Week3

Arbitrary argument lists

- Extra positional arguments collect into a tuple with `*args`
- **Order matters:** named params, then `*args`, then keyword-only params, then `**kwargs`
- Extra keyword arguments collect into a dict with `**kwargs`
- Unpack a list or dict into a call with `*` and `**` at the call site
- Great for wrapper functions that forward arguments to another call

```
# readable, PEP 8 style
def concat(*args, sep="/"):
    # args is a tuple
    return sep.join(args)
concat("a", "b", "c", sep="-")
# 'a-b-c'
```

Lambda expressions

- A **lambda** is a small anonymous function:
lambda arguments: expression
- Restricted to a single expression — no statements, no multiple lines
- Most useful as a short, throwaway function passed to another function
- Can reference variables from the enclosing scope

```
# equivalent to a def
square = lambda x: x ** 2

# sort by the second item
pairs.sort(key=lambda p: p[1])

# used inline, no def needed
sorted(words, key=lambda w: len(w))
```

Documentation strings

- A docstring is the first statement in a function, module, or class body
- **Convention:** a one-line summary, a blank line, then further detail
- Accessible at runtime via `func.__doc__` or `help(func)`
- Describe what a function does and its parameters — not how it works
- Triple-quoted, even for one-liners — that's the convention

```
# one-line docstring
def square(x):
    """Return the square of x."""
    return x ** 2
square.__doc__
# 'Return the square of x.'
```

Function annotations

- **An annotation** is optional metadata attached to a parameter or return value
- Written as param: type after the name, and -> type before the colon
- Purely informational
 - Python does not enforce or check them
- Stored in the function's `__annotations__` dictionary

```
# annotated function signature
def greet(name: str, times: int = 1) -> str:

# not enforced — just documentation
    return (name + "! ") * times

# inspect the annotations
greet.__annotations__
```

Tools for working with lists

- `map(func, iterable)` applies a function to every item, lazily
- **`filter(func, iterable)`** keeps only the items where `func` returns true
- List comprehensions are often more readable than `map/filter`
- `sorted(iterable, key=...)` sorts by a computed key, non-destructively
- `reversed()`, `enumerate()`, and `zip()` round out the toolkit

```
# map/filter vs. comprehension
squares = list(map(lambda x: x**2,
nums))
evens = [x for x in nums if x % 2 == 0]
# sort by a key, non-destructively
by_len = sorted(words, key=len)
# nums, words unchanged
```

WRAP-UP

Before next session

- Write one function of your own using `*args` or `**kwargs`
- Rewrite three list comprehensions as `map/filter` calls, and compare
- Add a docstring and type annotations to a function you wrote earlier
- Reach out to your class representative with any early questions